

1.9 Objets

Dans un langage de programmation, un **type** est

- Un ensemble de **valeurs**
- Des **opérations** sur ces valeurs

En plus des types primitifs, il est possible en C# de créer ses propres types. On appelle type construit un type non primitif, c'est-à-dire composé de types primitifs. Certains types construits sont fournis dans les bibliothèques du langage. Si ceux-là ne vous satisfont pas, vous avez la possibilité de créer vos propres types.

1.9.1 Création d'un type

Nous souhaitons créer un type **Point** dans R^2 . Chaque variable de ce type aura deux attributs, une abscisse et une ordonnée. Le type point se compose donc à partir de deux types flottants. Un type construit s'appelle une **classe**. On le définit comme suit :

```
class Point
{
    public double abscisse;
    public double ordonnee;
}
```

Les deux attributs d'un objet de type **Point** s'appelle aussi des **champs**. Une fois définie cette classe, le type **Point** est une type comme les autres, il devient donc possible d'écrire

```
Point p, q;
```

Cette instruction déclare deux variables **p** et **q** de type **Point**, ces variables s'appellent des **objets**. Chacun de ces objets a deux attributs auxquels on accède avec la notation pointée. Par exemple, l'abscisse du point **p** est **p.abscisse** et son ordonnée est **p.ordonnee**.

Voyons un exemple d'utilisation de cette classe :

```
class Point
{
    public double abscisse;
    public double ordonnee;
}

class MainClass
{
    public static void Main (string[] args)
    {
        Point p = new Point();
        p.ordonnee = 3;
        p.abscisse = 2;
        Console.WriteLine("p = (" + p.abscisse + ", " + p.ordonnee + ")");
    }
}
```

De la même façon que pour les tableaux, **p** n'est pas un point, mais une **variable contenant un point**. Par conséquent le **new Point();** est indispensable!

1.9.2 L'instanciation

Revenons sur cette histoire de **new**. Quand vous déclarez une variable non primitive, elle ne contient rien. La valeur utilisée dans les langages de programmation objet pour spécifier la valeur *rien* est **null**.

Par conséquent, ce n'est pas la déclaration qui permet de créer un objet mais l'utilisation de l'instruction **new**.

Le problème que cela pose s'observe sur l'exemple suivant :

```
class Point
{
    public double abscisse;
    public double ordonnee;
}

class MainClass
{
    public static void Main (string[] args)
    {
        Point p = new Point();
        p.ordonnee = 3;
        p.abscisse = 2;
        Point q = p;
        q.abscisse = 4;
        Console.WriteLine("p = (" + p.abscisse + ", " + p.ordonnee + ")");
    }
}
```

A votre avis, qu'affiche-t-il ? q est-il une copie de p ? Ou est-ce que q et p sont deux noms différents pour un même objet ?

Une variable de type `Point` n'est pas un point, mais l'adresse mémoire (identifiant, référence) d'un objet de type `Point`. Donc, `Point q = p;` crée une deuxième variable de type `Point`, mais qui contient la même adresse que p. Donc p et q référencent le même objet, toute modification sur l'objet référencé par p s'observera aussi sur l'objet référencé par q.

Lorsqu'un même objet porte plusieurs noms différents, on parle d'**aliasing**. Il convient d'être prudent avec les langages qui permettent l'aliasing, car cela peut engendrer des bugs très difficiles à trouver.

1.9.3 Les méthodes

Non contents d'avoir défini ainsi un ensemble de valeurs, nous souhaiterions définir un ensemble d'opérations sur ces valeurs. Nous allons pour ce faire nous servir de **méthodes**. Une méthode est un sous-programme propre à chaque objet. C'est-à-dire dont le contexte d'exécution est délimité par un objet. Par exemple,

```
class Point
{
    public double abscisse;
    public double ordonnee;

    public void presenteToi()
    {
        Console.WriteLine("Je suis un point, mes coordonnées sont ("
            + abscisse + ", " + ordonnee + ")");
    }
}

class MainClass
{
    public static void Main (string[] args)
    {
        Point p = new Point();
    }
}
```

```

        p.ordonnee = 3;
        p.abscisse = 2;
        p.presenteToi();
    }
}

```

On remarque qu'une méthode fonctionne comme un sous-programme à une différence près : il faut omettre le **static**.

La méthode `presenteToi` s'invoque à partir d'un objet de type `Point`. La syntaxe est `p.presenteToi()` où `p` est de type `Point`. `p` est alors le contexte de l'exécution de `presenteToi` et les champs auquel accèdera cette méthode seront ceux de l'objet `p`. Si par exemple, on écrit `q.presenteToi()`, c'est `q` qui servira de contexte à l'exécution de `presenteToi`. Lorsque l'on rédige une méthode, l'objet servant de contexte à l'exécution de la méthode est appelé l'**objet courant**.

Il est aussi possible de modifier les valeurs des attributs depuis une méthode :

```

class Point
{
    public double abscisse;
    public double ordonnee;

    public void initCoord(double a, double o)
    {
        abscisse = a;
        ordonnee = o;
    }

    public void presenteToi()
    {
        Console.WriteLine("Je suis un point, mes coordonnées sont ("
            + abscisse + ", " + ordonnee + ")");
    }
}

class MainClass
{
    public static void Main (string[] args)
    {
        Point p = new Point();
        p.initCoord(3, 2);
        p.presenteToi();
    }
}

```

Pour aller dans les applications tordues de la programmation objet, une méthode peut prendre en paramètre un autre objet :

```

class Point
{
    public double abscisse;
    public double ordonnee;

    public void initCoord(double a, double o)
    {
        abscisse = a;
        ordonnee = o;
    }
}

```

```

    public void copyCoord(Point autre)
    {
        abscisse = autre.abscisse;
        ordonnee = autre.ordonnee;
    }

    public void presenteToi()
    {
        Console.WriteLine("Je suis un point, mes coordonnées sont ("
            + abscisse + ", " + ordonnee + ")");
    }
}

class MainClass
{
    public static void Main (string[] args)
    {
        Point p = new Point();
        p.initCoord(3, 2);
        Point q = new Point();
        q.copyCoord(p);
        q.ordonnee ++;
        p.presenteToi();
        q.presenteToi();
    }
}

```

La méthode `initCoord` recopie les coordonnées d'un autre objet de type `Point` (`autre` en l'occurrence) à l'intérieur du point courant.

Vous avez aussi le droit de mettre au point des fonctions qui retournent un objet :

```

class Point
{
    public double abscisse;
    public double ordonnee;

    public void initCoord(double a, double o)
    {
        abscisse = a;
        ordonnee = o;
    }

    public void copyCoord(Point autre)
    {
        abscisse = autre.abscisse;
        ordonnee = autre.ordonnee;
    }

    public Point copyPoint()
    {
        Point res = new Point();
        res.ordonnee = ordonnee;
        res.abscisse = abscisse;
    }
}

```

```

        return res;
    }

    public void presenteToi()
    {
        Console.WriteLine("Je suis un point, mes coordonnées sont ("
            + abscisse + ", " + ordonnee + ")");
    }
}

class MainClass
{
    public static void Main (string[] args)
    {
        Point p = new Point();
        p.initCoord(3, 2);
        Point q = p.copyPoint();
        q.ordonnee ++;
        p.presenteToi();
        q.presenteToi();
    }
}

```

Le méthode `copyPoint` crée un point et le retourne après y avoir recopié les attributs du point courant. Donc, `p` et `q` sont deux points distincts.

1.9.4 Le mot-clé `this`

Dans toute méthode, vous disposez d'une référence vers l'objets servant de contexte à l'exécution de la méthode, cette référence s'appelle `this`.

```

class Point
{
    public double abscisse;
    public double ordonnee;

    public void initCoord(double abscisse, double ordonnee)
    {
        this.abscisse = abscisse;
        this.ordonnee = ordonnee;
    }

    public void copyCoord(Point autre)
    {
        abscisse = autre.abscisse;
        ordonnee = autre.ordonnee;
    }

    public Point copyPoint()
    {
        Point res = new Point();
        res.copyCoord(this);
        return res;
    }
}

```

```

    public void presenteToi()
    {
        Console.WriteLine("Je suis un point, mes coordonnées sont ("
            + abscisse + ", " + ordonnee + ")");
    }
}

class MainClass
{
    public static void Main (string[] args)
    {
        Point p = new Point();
        p.initCoord(3, 2);
        Point q = p.copyPoint();
        q.ordonnee ++;
        p.presenteToi();
        q.presenteToi();
    }
}

```

Dans l'exemple ci-dessus, la méthode `initCoord` a été modifiée, les noms des paramètres formels sont les mêmes que les noms des champs. Le préfixe `this` permet d'utiliser des attributs et en l'absence de préfixe, C# choisit en priorité les variables locales. Dans la méthode `copyPoint`, le mot clé `this` est employé pour envoyer l'objet en courant en paramètre dans la méthode d'un autre point.